

ПРИЛОЖЕНИЕ Б

(Обязательное)

Программный код средства построения радиолокационных изображений для маршрутного режима радиолокатора космического базирования

Файл интерфейса *main_start_detail.py*

```
from PySide6.QtUiTools import QUiLoader
from PySide6.QtWidgets import (QApplication,
                                QPushButton,
                                QCheckBox,
                                QGroupBox,
                                QDoubleSpinBox,
                                QSpinBox,
                                QLabel,
                                QTextBrowser,
                                QRadioButton,
                                QFileDialog,
                                QComboBox,
                                QMainWindow)

from PySide6.QtCore import Slot
from PySide6.QtGui import QPixmap
from v2.calc_rli import calc_rli
from v2.GetFilesPath import getFilesPath

app = QApplication([])
window = QMainWindow()
ui_file = "UI/detail.ui"
loader = QUiLoader()
ui = loader.load(ui_file)

# Селекторы
button_start = ui.findChild(QPushButton, "Start")
button_uploadFile = ui.findChild(QPushButton, "upload")

QTypeWinDn = ui.findChild(QGroupBox, 'TypeWinDn')
QTypeWinDp = ui.findChild(QGroupBox, 'TypeWinDp')

pathUi = ui.findChild(QLabel, 'Path')

selectedTypeWinDnArr = QTypeWinDn.findChildren(QRadioButton)
selectedTypeWinDpArr = QTypeWinDp.findChildren(QRadioButton)

comboBox_Dp = ui.findChild(QComboBox, 'comboBox_Dp')
comboBox_Dn = ui.findChild(QComboBox, 'comboBox_Dn')

outPutLabel = ui.findChild(QTextBrowser, 'Output')

@Slot()
def QPrint(value):
    text = outPutLabel.toPlainText()
    outPutLabel.setText(f"{text}\n{value}")

def button_start_clicked():
    # Default values
    TypeWinDn = 1
    TypeWinDp = 1
```

```

# Ищем режим TypeWinDn
for i in range(len(selectedTypeWinDnArr)):
    if selectedTypeWinDnArr[i].isChecked():
        TypeWinDn = int(selectedTypeWinDnArr[i].objectName()[1:])

# Ищем режим TypeWinDp
for i in range(len(selectedTypeWinDpArr)):
    if selectedTypeWinDpArr[i].isChecked():
        TypeWinDp = int(selectedTypeWinDnArr[i].objectName()[1:])

# Ищем пути к файлам
ConsortPath, ModelDatePath, Yts1Path, Yts2Path =
getFilePath(str(pathUi.text()))

returnedValues = {
    'Kss': int(ui.findChild(QSpinBox, "Kss").text()),
    'dxsint': float(ui.findChild(QDoubleSpinBox,
"dxsint").text().replace(',', '.')),
    'dysint': float(ui.findChild(QDoubleSpinBox,
"dysint").text().replace(',', '.')),
    'StepBright': float(ui.findChild(QDoubleSpinBox,
"StepBright").text().replace(',', '.')),
    'Nxsint': int(ui.findChild(QSpinBox, "Nxsint").text()),
    'Nysint': int(ui.findChild(QSpinBox, "Nysint").text()),
    'Tsint': float(ui.findChild(QDoubleSpinBox,
"Tsint").text().replace(',', '.')),
    'tauRli': float(ui.findChild(QDoubleSpinBox,
"tauRli").text().replace(',', '.')),
    'RegimRsa': 1,
    'TypeWinDp': TypeWinDp,
    'TypeWinDn': TypeWinDn,
    'isGPU': ui.findChild(QCheckBox, 'GPU').isChecked(),
    'FlagViewSignal': ui.findChild(QCheckBox,
'FlagViewSignal').isChecked(),
    'FlagWriteRli': ui.findChild(QCheckBox, 'FlagWriteRli').isChecked(),
    'ConsortPath': ConsortPath,
    'ModelDatePath': ModelDatePath,
    'Yts1Path': Yts1Path,
    'Yts2Path': Yts2Path
}

print('Запуск расчёта...')
calc_rli(returnedValues, QPrint)

```

```
QPrint('Hello world!')
```

```

def open_file():
    file_dialog = QFileDialog()
    file_path, _ = file_dialog.getOpenFileName(window, "Выберите файл")

    if file_path:
        pathUi.setText(file_path)
        ConsortPath, ModelDatePath, Yts1Path, Yts2Path =
getFilePath(str(pathUi.text()))
        ModelDateContent = []
        with open(ModelDatePath, 'r') as f:
            while True:
                # считываем строку
                line = f.readline()
                if not line:
                    break

```

```

        ModelDateContent.append(float(line))
        ModelDateLabel = ui.findChild(QTextBrowser, 'textBrowserModalDate')
        RowsAndCulCount = ui.findChild(QTextBrowser, 'RowsAndCulCount')
        formData = f"высота орбиты РСА = {format(ModelDateContent[0])}\n" \
            f"длина волны = {format(ModelDateContent[1])}\n" \
            f"ширина главного лепестка ДН по азимуту = \
{format(ModelDateContent[2])}\n" \
            f"ширина главного лепестка ДН по углу места = \
{format(ModelDateContent[3])}\n" \
            f"длительность импульса = {format(ModelDateContent[4])}\n" \
            f"период повторения = {format(ModelDateContent[5])}\n" \
            f"ширина спектра сигнала = \
{format(ModelDateContent[6])}\n" \
            f"частота дискретизации = {format(ModelDateContent[7])}\n" \
            f"широта центра участка синтезирования = \
{format(ModelDateContent[8])}\n" \
            f"параметр согласованного фильтра  $bzc=\pi \cdot df/T0$  = \
{format(ModelDateContent[10])}\n" \
            f"время задержки записи по отношению к началу периода = \
{format(ModelDateContent[11])}\n" \
            f"число отсчетов по быстрому времени = \
{format(ModelDateContent[12])}\n" \
            f"число периодов повторения = \
{format(ModelDateContent[13])}\n" \
            f"число приемных каналов = \
{format(ModelDateContent[14])}\n" \
            f"расстояние между фазовыми центрами приемных каналов = \
{format(ModelDateContent[15])}\n" \
            f"скорость РСА = {format(ModelDateContent[16])}\n" \
            f"время синтезирования = {format(ModelDateContent[17])}\n" \
            f"момент начала получения траекторного сигнала = \
{format(ModelDateContent[18])}\n" \
            f"дискретность данных в консорт-файле по координатам = \
{format(ModelDateContent[19])}\n" \
            f"дискретность данных в консорт-файле по скорости = \
{format(ModelDateContent[20])}\n" \
            f"дискретность данных в консорт-файле по углам = \
{format(ModelDateContent[21])}"
        ModelDateLabel.setText(formData)
        RowsAndCulCount.setText(
            f"Размер входного массива - {int(ModelDateContent[12])} x \
{int(ModelDateContent[13])}\n " \
            f"type Int16\n" \
            f"Размер файла Yts1 - {(int(ModelDateContent[12]) * \
int(ModelDateContent[13]) * 2 * 2) / 1024 / 1024} Мб")
        fileNameLabel = ui.findChild(QLabel, 'File_name')
        fileNameLabel.setText(str(file_path).split("/")[-1])
        QPrint(f"Выбран файл: {file_path}")

button_uploadFile.clicked.connect(open_file)
button_start.clicked.connect(button_start_clicked)

ui.show()
app.exec()

```

Файл подготовительных расчетов *calc_rli.py*:

```
import math

import numpy as np
import matplotlib.pyplot as plt

from v2.functions import Win, ChirpSig
from v1.utils import time_of_function
from v2.route_mode import route_big_cycle1, route_big_cycle2
from v2.detail_mode import detail_big_cycle1, detail_big_cycle2
from v2.route_mode_gpu import gpu_route_big_cycle1, gpu_route_big_cycle2
from v2.detail_mode_gpu import gpu_detail_big_cycle1, gpu_detail_big_cycle2

# МАРШРУТНЫЙ РЕЖИМ: построение РЛИ в координатах (широта/долгота) при
# увеличенной частоты дискретизации.
# Вход:
# файл с именем app.fileIsx с параметрами моделирования
# бинарные файлы с именами app.fileTS1, fileTS2 с траекторными сигналами
# одного (Nrch=1) или двух (Nrch=2) интерферометрических приемных каналов
# файл с именем app.FileConsort с координатами РСА по периодам повторения
# При ВПО проводится увеличение частоты дискретизации траекторного сигнала
# в Kss раз за счет дополнения спектра нулями справа, вычисляется ИХ и КЧХ
# согласованного фильтра ВПО при увеличенной частоте дискретизации с учетом
# типа TypeWinDn оконной функции по дальности, перемножение спектра сигнала
# и КЧХ фильтра и обратное ДПФ.
# При МПО проводится синтез для увеличенной частоты дискретизации, для чего
# с заданной дискретностью просматриваются все точки заданной зоны
# синтезирования по широте и долготе при нулевой высоте точки на Земле.
# Для каждой точки вычисляется момент времени прохождения траверса, когда
# азимут точки равен нулю, интервал синтезирования симметричен
# относительно траверса, центральный период повторения равен q0
# для каждого периода повторения в зоне синтезирования от q1=q0-Tsint/Tr/2
# до q2=q0+Tsint/Tr/2 проводится вычисление дальности до точки, индекс n
# положения сжатого сигнала по быстрому времени и суммирование этих
# сигналов на всем интервале синтезирования с компенсацией изменения фазы.
# Расчеты проводятся для одного или двух приемных каналов с построением
# разности фаз сформированных РЛИ или разностного РЛИ

##### ПАРАМЕТРЫ ОБРАБОТКИ #####
# оконные функции:
# 1 - прямоугольное окно
# 2 - косинус на пьедестале, при delta=0.54 - Хэмминга (-42.7 дБ)
# 3 - косинус квадрат на пьедестале
# 4 - Хэмминга (-42.7 дБ)
# 5 - Хэннинга в третьей степени на пьедестале (-39.3 дБ)
# 6 - Хэннинга в четвертой степени на пьедестале (-46.7 дБ)
# 7,8,9 - Кайзера-Бесселя для alfa=2.7 3.1 3.5 (-62.5 -72.1 -81.8 дБ)
# 10 - Блекмана-Херриса (-92 дБ)

# вариант задания исходных данных через форму
def calc_rli(client_values, QPrint):
    ##### Считывание данных из UI #####
    Kss = client_values['Kss'] # коэффициент передискретизации
    dxsint = client_values['dxsint'] # дискретность зоны синтезирования по
    Oх
    dysint = client_values['dysint'] # дискретность зоны синтезирования по
    Oу
    StepBright = client_values['StepBright'] # показатель степени при
```

отображении

```
Nxsint = client_values['Nxsint'] # число точек по оси Oх (долготе)
Nysint = client_values['Nysint'] # число точек по оси Oу (широте)
Tsint = client_values['Tsint'] # время синтезирования
tauRli = client_values['tauRli']
RegimRsa = client_values['RegimRsa'] # режим радиолокационной съемки 1 -
детальный, 2 - маршрутный
TypeWinDp = client_values['TypeWinDp'] # тип оконной функции при сжатии
по поперечной дальности
TypeWinDn = client_values['TypeWinDn'] # тип оконной функции при сжатии
по наклонной дальности
GPUCalculationFlag = client_values['isGPU']
ConsortPath = client_values['ConsortPath']
ModelDatePath = client_values['ModelDatePath']
Yts1Path = client_values['Yts1Path']
Yts2Path = client_values['Yts2Path']
FlagViewSignal = client_values['FlagViewSignal'] # флаг отображения
сигналов в ходе расчетов
FlagWriteRli = client_values['FlagWriteRli']
NumRli = 1 # номер РЛИ в последовательности ***
```

```
##### СЧИТЫВАНИЕ ИСХОДНЫХ ДАННЫХ
#####
```

```
# параметры Земли
# считывание параметров моделирования из консорт-файла параметров PCA
ModelDate=[]
with open(ModelDatePath, 'r') as f:
    while True:
        # считываем строку
        line = f.readline()
        if not line:
            break
        ModelDate.append(float(line))
    # прерываем цикл, если строка пустая
```

```
Hrsa = ModelDate[0] # высота орбиты PCA
lamda = ModelDate[1] # длина волны
alfa05Ar = ModelDate[2] # ширина главного лепестка ДН по азимуту
teta05Ar = ModelDate[3] # ширина главного лепестка ДН по углу места
T0 = ModelDate[4] # длительность импульса
Tr = ModelDate[5] # период повторения
df = ModelDate[6] # ширина спектра сигнала
Fs = ModelDate[7] # частота дискретизации
fizt0 = ModelDate[8] # широта центра участка синтезирования
betazt0 = ModelDate[9] # долгота центра участка синтезирования
bzc = ModelDate[10] # параметр согласованного фильтра  $bzc = \pi * df / T0$ 
t_r_w = ModelDate[11] # время задержки записи по отношению к началу
периода
N = int(ModelDate[12]) # число отсчетов по быстрому времени
Q = int(ModelDate[13]) # число периодов повторения
Nrch = int(ModelDate[14])
Nrch = 1 # число приемных каналов
Lrch = ModelDate[15] # расстояние между фазовыми центрами приемных
каналов
Vrsa = ModelDate[16] # скорость PCA
# Tsint = ModelDate[17] # время синтезирования ?
Tst0 = ModelDate[18] # момент начала получения траекторного сигнала
dxConsort = ModelDate[19] # дискретность данных в консорт-файле по
координатам
dVxConsort = ModelDate[20] # дискретность данных в консорт-файле по
скорости
dugConsort = ModelDate[21] # дискретность данных в консорт-файле по
углам
```

```

# считывание координат PCA из консорт-файла размерностью Q*14
XYZ_rsa_ts = np.genfromtxt(ConsortPath, dtype=float, delimiter=' ')
# считывание траекторного сигнала из файла
with open(Yts1Path, 'rb') as file:
    Yts = np.fromfile(file, dtype=np.int16).reshape((Q, 2 * N)).T

# формируем комплексный массив и преобразуем его к целым числам
Yts1r = Yts[:N, :Q] + 1j * Yts[N:2 * N, :Q]

if Nrch == 2:
    with open(Yts2Path, 'rb') as file:
        Yts = np.fromfile(file, dtype=np.int16).reshape((Q, 2 * N)).T
    # формируем комплексный массив и преобразуем его к целым числам
    Yts2r = Yts[:N, :Q] + 1j * Yts[N:2 * N, :Q]

del Yts

# КОНСТАНТЫ
Rz = 6371000 # радиус Земли
wz = 7.2921158553e-05 # угловая скорость Земли
Tz = 2 * np.pi / wz # период вращения Земли - солнечные сутки
speedOfL = 3 * 10 ** 8 # скорость света
e = 2.71828

qst = int(tauRli / Tr) + 1 # номер периода повторение с которого
начинаем расчет для детального режима

##### ОСНОВНЫЕ ВЫЧИСЛЕНИЯ
#####
# вычисление внутрипериодных спектров сигналов приемных каналов

Y01 = np.fft.fft(Yts1r, axis=0)
if Nrch == 2:
    Y02 = np.fft.fft(Yts2r, axis=0)
del Yts2r

del Yts1r
##### ВПО с передискретизацией
#####

# вычисление КЧХ фильтра с увеличенным числом отсчетов с учетом оконной
функции
h0ss = np.zeros(N * Kss, np.complex64)
for n in range(int(T0 * Fs * Kss + 1)):
    h0ss[n] = np.conj(ChirpSig(T0 - (n) / (Fs * Kss), T0, bzc)) * Win((T0
- (n) / (Fs * Kss)) / T0 - 0.5, 0, TypeWinDn)

Gh0ss = np.fft.fft(h0ss)
Gh0ss = Gh0ss / math.sqrt(N * Kss)

del h0ss

# передискретизация по наклонной дальности - дополнение спектра нулями
# новый вариант передискретизации - просто добавляем нули

@time of function
def oversampling(N, Q, Kss, Y0X, Gh0ss):
    Y01ss = np.empty((Kss * N, Q), np.complex64)
    Y01ss[:N, :Q] = Y0X[:N, :Q]

    # сжатие по дальности
    Goutss = Y01ss * Gh0ss[:, np.newaxis] # Векторизованная операция

    return Goutss

```

```

# 4 секунды копирование и перемножение матриц 16 - обратное БПФ

np.fft.ifft = time_of_function(np.fft.ifft)

Goutss = oversampling(N, Q, Kss, Y01, Gh0ss)
if GPUCalculationFlag:
    from v2.cupy_functions import cuiffft
    Uout01ss = cuiffft(Goutss) * np.sqrt(N * Kss)
else:
    Uout01ss = np.fft.ifft(Goutss, axis=0) * np.sqrt(N * Kss)

if Nrch == 2:
    # второй канал
    # новый вариант передискретизации - просто добавляем нули
    Goutss = oversampling(N, Q, Kss, Y02, Gh0ss)
    if GPUCalculationFlag:
        from v2.cupy_functions import cuiffft
        Uout02ss = cuiffft(Goutss) * np.sqrt(N * Kss)
    else:
        Uout02ss = np.fft.ifft(Goutss, axis=0) * np.sqrt(N * Kss)

del Gh0ss
# отображение сигналов до после ВПО при установленном флаге отображения
# if FlagViewSignal == 1: # 13 секунд !!!
#     # График 1
#     fig1 = plt.figure()
#     n = np.arange(1, N * Kss + 1)
#     plt.plot(n, np.abs(Uout01ss[n - 1, 0]), n, np.abs(Uout01ss[n - 1, Q
// 2]), n, np.abs(Uout01ss[n - 1, Q - 1]),
#             linewidth=0.5)
#     plt.grid(True)
#     plt.title('Сигналы после ВПО')
#
#     # График 2
#     fig2 = plt.figure()
#
#     Vrli0 = (np.abs(Y01.T) ** StepBright)
#     Vrli0 = Vrli0 / np.max(np.max(Vrli0))
#     Vrli0 = np.flipud(Vrli0)
#
#     plt.subplot(2, 1, 1)
#     plt.imshow(Vrli0)
#     plt.xlabel('Наклонная дальность')
#     plt.ylabel('Период повторения')
#     plt.title('Сигналы на входе ВПО')
#     Vrli0 = (np.abs(Uout01ss.T) ** StepBright)
#     Vrli0 = Vrli0 / np.max(np.max(Vrli0))
#     Vrli0 = np.flipud(Vrli0)
#
#     plt.subplot(2, 1, 2)
#     plt.imshow(Vrli0)
#     plt.xlabel('Наклонная дальность')
#     plt.ylabel('Период повторения')
#     plt.title('Сигналы на выходе ВПО')

# ##### сжатие в координатах (широта/долгота) Backprojection
#####
# отсчеты накопленного комплексного РЛИ-1
Zxy1 = np.zeros((Nxsint, Nysint), np.complex64)
if Nrch == 2:
    Zxy2 = np.zeros((Nxsint, Nysint), np.complex64)

# подготовка значений оконной функции
Nr = int(Tsint / Tr) # число периодов повторения на интервале

```

```

синтезирования
WinSampl = np.ones(Nr + 1)
for q in range(Nr + 1):
    WinSampl[q] = Win((q + 1 - Nr / 2) / Nr, 0, TypeWinDp) # отсчеты
оконной функции

if RegimRsa == 2: # маршрутный режим
    # основной расчетный цикл ДЛЯ МАКСИМАЛЬНОГО РАСПАРАЛЛЕЛИВАНИЯ
    if GPUCalculationFlag:
        if Nrch == 1:
            Zxy1 = gpu_route_big_cycle1(Zxy1, Nxsint, Nysint, Uout01ss,
                                         dxsint, dysint, fizt0,
                                         Rz, betazt0, Tst0, Q, Tr,
                                         XYZ_rsa_ts, dxConsort,
                                         dVxConsort, Tz, Vrsa,
                                         Hrsa, dugConsort, Tsint, Lrch,
                                         speedOfL, t_r_w, Kss, Fs,
                                         lamda, WinSampl, e, T0)
        if Nrch == 2:
            Zxy1, Zxy2 = gpu_route_big_cycle2(Zxy1, Zxy2, Nxsint, Nysint,
                                                Uout01ss, Uout02ss, dxsint,
                                                dysint, fizt0, Rz, betazt0,
                                                Tst0, Q, Tr, XYZ_rsa_ts,
                                                dxConsort, dVxConsort, Tz,
                                                Vrsa, Hrsa, dugConsort,
                                                Tsint, Lrch, speedOfL,
                                                t_r_w, Kss, Fs, lamda,
                                                WinSampl, e, T0)
    else:
        if Nrch == 1:
            Zxy1 = route_big_cycle1(Zxy1, Nxsint, Nysint, Uout01ss,
                                     dxsint, dysint, fizt0, Rz, betazt0,
                                     Tst0, Q, Tr, XYZ_rsa_ts, dxConsort,
                                     dVxConsort, Tz, Vrsa, Hrsa,
                                     dugConsort, Tsint, Lrch, speedOfL,
                                     t_r_w, Kss, Fs, lamda, WinSampl, e,
                                     T0)
        if Nrch == 2:
            Zxy1, Zxy2 = route_big_cycle2(Zxy1, Zxy2, Nxsint, Nysint,
                                            Uout01ss, Uout02ss, dxsint,
                                            dysint, fizt0, Rz, betazt0,
                                            Tst0, Q, Tr, XYZ_rsa_ts,
                                            dxConsort, dVxConsort, Tz,
                                            Vrsa, Hrsa, dugConsort, Tsint,
                                            Lrch, speedOfL, t_r_w, Kss, Fs,
                                            lamda, WinSampl, e, T0)

if RegimRsa == 1: # детальный режим
    # подготовка исходных данных для расчета
    Tst = Tst0 + (qst - 1) * Tr # момент начала
    Inabl = Nr # число точек
    Tnabl = Nr * Tr # время общее наблюдения
    q1 = qst # номер первого периода повторения
    q2 = qst + Nr - 1 # номер последнего периода повторения
    if q2 > Q:
        q2 = Q
    print('Интервал синтеза выходит за пределы траекторного
сигнала')
    QPrint('Интервал синтеза выходит за пределы траекторного
сигнала')

    # подготовительные операции для аппроксимации дальности
    qq = np.empty(5, dtype=np.int32)
    tt = np.empty(5)
    Q_cons_0 = qst

```



```

qq[0] = Q_cons_0 - 1
tt[0] = 0
qq[1] = Q_cons_0 - 1 + int(Inabl / 4) - 1
tt[1] = Tnabl / 4
qq[2] = Q_cons_0 - 1 + int(Inabl / 2) - 1
tt[2] = Tnabl / 2
qq[3] = Q_cons_0 - 1 + int(3 * Inabl / 4) - 1
tt[3] = 3 * Tnabl / 4
qq[4] = Q_cons_0 - 1 + Inabl - 1
tt[4] = Tnabl
sumtt = np.sum(tt)
sumtt2 = np.sum(tt ** 2)
sumtt3 = np.sum(tt ** 3)
sumtt4 = np.sum(tt ** 4)
sumtt5 = np.sum(tt ** 5)
sumtt6 = np.sum(tt ** 6)

# основной расчетный цикл ДЛЯ МАКСИМАЛЬНОГО РАСПАРАЛЛЕЛИВАНИЯ
if GPUCalculationFlag:
    if Nrch == 1:
        Zxy1 = gpu_detail_big_cycle1(Zxy1, Nxsint, Nysint, Uout01ss,
                                      dxsint, dysint, fizt0,
                                      Rz, betazt0, Tr, XYZ_rsa_ts,
                                      dxConsort, Tz, Vrsa,
                                      tauRli, Inabl, qq, tt,
                                      Lrch, speedOfL, t_r_w, Kss, Fs,
                                      lamda, WinSampl, e, T0,
                                      sumtt, sumtt2, sumtt3, sumtt4,
                                      sumtt5, sumtt6, q1, q2, Tst)
    if Nrch == 2:
        Zxy1, Zxy2 = gpu_detail_big_cycle2(Zxy1, Zxy2, Nxsint,
                                             Nysint, Uout01ss, Uout02ss,
                                             dxsint, dysint, fizt0,
                                             Rz, betazt0, Tr,
                                             XYZ_rsa_ts, dxConsort, Tz,
                                             Vrsa, tauRli, Inabl, qq,
                                             tt, Lrch, speedOfL, t_r_w,
                                             Kss, Fs, lamda, WinSampl,
                                             e, T0, sumtt, sumtt2,
                                             sumtt3, sumtt4, sumtt5,
                                             sumtt6, q1, q2, Tst)
else:
    if Nrch == 1:
        Zxy1 = detail_big_cycle1(Zxy1, Nxsint, Nysint, Uout01ss,
                                  dxsint, dysint, fizt0,
                                  Rz, betazt0, Tr, XYZ_rsa_ts,
                                  dxConsort, Tz, Vrsa, tauRli,
                                  Inabl, qq, tt, Lrch, speedOfL,
                                  t_r_w, Kss, Fs, lamda, WinSampl, e,
                                  T0, sumtt, sumtt2, sumtt3, sumtt4,
                                  sumtt5, sumtt6, q1, q2, Tst)
    if Nrch == 2:
        Zxy1, Zxy2 = detail_big_cycle2(Zxy1, Zxy2, Nxsint, Nysint,
                                         Uout01ss, Uout02ss, dxsint,
                                         dysint, fizt0, Rz, betazt0,
                                         Tr, XYZ_rsa_ts, dxConsort, Tz,
                                         Vrsa, tauRli, Inabl, qq, tt,
                                         Lrch, speedOfL, t_r_w, Kss, Fs,
                                         lamda, WinSampl, e, T0, sumtt,
                                         sumtt2, sumtt3, sumtt4,
                                         sumtt5, sumtt6, q1, q2, Tst)

if FlagViewSignal == 1:
    # Трехмерное РЛИ-1

```

```

fig = plt.figure(figsize=(12.8, 9.6))
ax = fig.add_subplot(111, projection='3d')
X, Y = np.meshgrid(np.arange(Zxy1.shape[1]),
np.arange(Zxy1.shape[0]))
ax.plot_surface(X, Y, np.abs(Zxy1) ** StepBright, cmap='pink')
ax.view_init(10, -60)
ax.set_xlabel('Широта')
ax.set_ylabel('Долгота')
ax.set_title('Трехмерное РЛИ-1. NumRli=' + str(NumRli) + ', степень='
+ str(StepBright))

# Яркое РЛИ-1
Vrli0 = np.abs(Zxy1) ** StepBright
Vrli0 = Vrli0 / np.max(Vrli0)
Vrli0 = np.flipud(Vrli0)

fig = plt.figure(figsize=(12.8, 9.6))
plt.imshow(Vrli0, cmap='gist_gray')
plt.xlabel('Широта')
plt.ylabel('Долгота')
plt.title('Яркое РЛИ-1. NumRli=' + str(NumRli) + ', степень=' +
str(StepBright))

if Nrch == 2:
    # Трехмерное РЛИ-2
    fig = plt.figure(figsize=(12.8, 9.6))
    ax = fig.add_subplot(111, projection='3d')
    ax.plot_surface(X, Y, np.abs(Zxy2) ** StepBright, cmap='pink')
    ax.view_init(10, -60)
    ax.set_xlabel('Широта')
    ax.set_ylabel('Долгота')
    ax.set_title('Трехмерное РЛИ-2. NumRli=' + str(NumRli) + ',
степень=' + str(StepBright))

    # Яркое РЛИ-2
    Vrli0 = np.abs(Zxy2) ** StepBright
    Vrli0 = Vrli0 / np.max(Vrli0)
    Vrli0 = np.flipud(Vrli0)

    fig = plt.figure(figsize=(12.8, 9.6))
    plt.imshow(Vrli0, cmap='gist_gray')
    plt.xlabel('Широта')
    plt.ylabel('Долгота')
    plt.title('Яркое РЛИ-2. NumRli=' + str(NumRli) + ', степень='
+ str(StepBright))

plt.show()

# ВЫВОД МАКСИМУМОВ И РАЗНОСТИ ФАЗ
Zxy1max = np.max(np.abs(Zxy1))
nqmax = np.argmax(np.abs(Zxy1))
nmax1 = nqmax % Nxsint
qmax1 = nqmax // Nxsint + 1
print(
    f'Zxy1({nmax1},{qmax1})={Zxy1max} x={(-Nxsint + 1) / 2 + nmax1 - 1 *
dxsint} y={(-Nysint + 1) / 2 + qmax1 - 1 * dxsint}')
QPrint(
    f'Zxy1({nmax1},{qmax1})={Zxy1max} x={(-Nxsint + 1) / 2 + nmax1 - 1 *
dxsint} y={(-Nysint + 1) / 2 + qmax1 - 1 * dxsint}')
if Nrch == 2:
    Zxy2max = np.max(np.abs(Zxy2))
    nqmax = np.argmax(np.abs(Zxy2))
    nmax2 = nqmax % Nxsint
    qmax2 = nqmax // Nxsint + 1

```

```

print(
    f'Zxy2({nmax2},{qmax2})={Zxy2max} x={(-Nxsint + 1) / 2 + nmax2 -
1 * dxsint} y={(-Nysint + 1) / 2 + qmax2 - 1 * dxsint}')
QPrint(
    f'Zxy2({nmax2},{qmax2})={Zxy2max} x={(-Nxsint + 1) / 2 + nmax2 -
1 * dxsint} y={(-Nysint + 1) / 2 + qmax2 - 1 * dxsint}')
print(f'Разность фаз={np.angle(Zxy1[nmax1, qmax1]) -
np.angle(Zxy2[nmax2, qmax2])}')
QPrint(f'Разность фаз={np.angle(Zxy1[nmax1, qmax1]) -
np.angle(Zxy2[nmax2, qmax2])}')

# Оценка отношения сигнал/шум
Psh = np.sum(np.abs(Zxy1) ** 2) / float(Nxsint) / float(Nysint)
print(f'ОШ максимальное={np.max(np.abs(Zxy1) ** 2) / Psh}')
QPrint(f'ОШ максимальное={np.max(np.abs(Zxy1) ** 2) / Psh}')
t = np.datetime64('now')
print(f'Завершение РЛИ ху {t}')
QPrint(f'Завершение РЛИ ху {t}')
# Запись сформированного РЛИ в файл
# Сначала записывается массив N*Q реальных значений, потом массив мнимых
значений;
# общая размерность 2*2N*Q байт
if FlagWriteRli == 1:
    print('Запись РЛИ в файл')
    QPrint('Запись РЛИ в файл')
    # первый канал
    Zxy = np.zeros((2 * Nxsint, Nysint), dtype=np.float64)
    Zxy[:Nxsint, :] = np.real(Zxy1[:Nxsint, :Nysint])
    Zxy[Nxsint:2 * Nxsint, :] = np.imag(Zxy1[:Nxsint, :Nysint])
    # формируем имя файла путем замены 'Yts1' в начале на Rli1 с теми же
значениями даты и времени
    filename = 'Yts1-14-May-2023 19:38:28'
    filename = Yts1Path.split('/')[ -1]
    filename = 'RL11' + filename[4:]
    fileID = open(filename, 'wb')
    if fileID.closed:
        raise Exception('Текущая папка защищена. Смените текущую папку')
    fileID.write(Zxy.tobytes())
    fileID.close()
    # второй канал
    if Nrch == 2:
        # первый канал
        Zxy[:Nxsint, :] = np.real(Zxy2[:Nxsint, :Nysint])
        Zxy[Nxsint:2 * Nxsint, :] = np.imag(Zxy2[:Nxsint, :Nysint])
        # формируем имя файла путем замены 'Yts1' в начале на Rli1 с теми
же значениями даты и времени
        filename = 'Yts1-14-May-2023 19:38:28'
        filename = 'RL12' + filename[4:]
        fileID = open(filename, 'wb')
        if fileID.closed:
            raise Exception('Текущая папка защищена. Смените текущую
папку')

        fileID.write(Zxy.tobytes())
        fileID.close()

print('РЛИ записаны в файл')
QPrint('РЛИ записаны в файл')
t = np.datetime64('now')
print(f'Завершение РЛИ ху {t}')
QPrint(f'Завершение РЛИ ху {t}')

```

Файл основных вспомогательных функций *functions.py*:

```

import numpy as np
import math
from scipy.special import iv

def ChirpSig(t, tx, b):
    win = np.logical_and(t >= 0, t <= tx)
    comp = win * np.exp(1j * complex(b * (t ** 2)))
    return comp

def Win(tau, delta, n):
    W = 0

    # функции - типовые окна: delta - уровень пьедестала; tau=(t/T0-1/2)
    # 1 - прямоугольное окно
    Wrect = lambda tau, delta: 1

    # 2 - косинус на пьедестале, при delta=0.54 - Хэмминга (-42.7 дБ)
    Wcos = lambda tau, delta: delta + (1 - delta) * math.cos(np.pi * tau)

    # 3 - косинус квадрат на пьедестале
    Wcos2 = lambda tau, delta: delta + (1 - delta) * (math.cos(np.pi * tau)
** 2)

    # 4 - Хэмминга (-42.7 дБ)
    Whemming = lambda tau, delta: 0.54 + 0.46 * math.cos(np.pi * tau)

    # 5 - Хэннинга в третьей степени на пьедестале (-39.3 дБ)
    Whenning3 = lambda tau, delta: delta + (1 - delta) * (math.cos(np.pi *
tau) ** 3)

    # 6 - Хэннинга в четвертой степени на пьедестале (-46.7 дБ)
    Whenning4 = lambda tau, delta: delta + (1 - delta) * (math.cos(np.pi *
tau) ** 4)

    # 7,8,9 - Кайзера-Бесселя для alfa=2.7; 3.1; 3.5 (-62.5; -72.1; -81.8 дБ)
    Wkb27 = lambda tau, delta: iv(0, 2.7 * np.pi * np.sqrt(1 - (2 * tau) **
2)) / iv(0, 2.7 * np.pi)
    Wkb31 = lambda tau, delta: iv(0, 3.1 * np.pi * np.sqrt(1 - (2 * tau) **
2)) / iv(0, 3.1 * np.pi)
    Wkb35 = lambda tau, delta: iv(0, 3.5 * np.pi * np.sqrt(1 - (2 * tau) **
2)) / iv(0, 3.5 * np.pi)

    # 10 - Блэкмана-Херриса (-92 дБ)
    Wbx = lambda tau, delta: 0.35875 + 0.48829 * math.cos(2 * np.pi * tau) +
0.14128 * math.cos(
    4 * np.pi * tau) + 0.01168 * math.cos(6 * np.pi * tau)

    if n == 1:
        W = Wrect(tau, delta)
    elif n == 2:
        W = Wcos(tau, delta)
    elif n == 3:
        W = Wcos2(tau, delta)
    elif n == 4:
        W = Whemming(tau, delta)
    elif n == 5:
        W = Whenning3(tau, delta)
    elif n == 6:
        W = Whenning4(tau, delta)
    elif n == 7:

```

```

        W = Wkb27(tau, delta)
    elif n == 8:
        W = Wkb31(tau, delta)
    elif n == 9:
        W = Wkb35(tau, delta)
    elif n == 10:
        W = Wbx(tau, delta)

```

```

return W

```

Файл основного алгоритма на *CUDA detail_mode_gpu.py*:

```

import math
import numpy as np
from numba import cuda
from v1.utils import time_of_function, time_of_function_compile
from v2.jit_functions import inverse_matrix_cuda4, dot_matrix_cuda,
flip_and_transpose_cuda

@cuda.jit
def kernel_2d_array_1(Zxyl, Nxsint, Nysint, Uout0lss, dxsint, dysint, fizt0,
                    Rz, betazt0, Tr, XYZ_rsa_ts, dxConsort, Tz, Vrsa,
                    tauRli, Inabl, qq, tt,
                    Lrch, speedOfL, t_r_w, Kss, Fs, lamda, WinSampl, e, T0,
                    sumtt, sumtt2, sumtt3, sumtt4, sumtt5, sumtt6, q1, q2,
                    Tst):
    nx, ny = cuda.grid(2)
    if nx < Zxyl.shape[0] and ny < Zxyl.shape[1]:
        # координаты текущей точки наблюдения
        xt = -(Nxsint - 1) / 2 + nx * dxsint
        yt = -(Nysint - 1) / 2 + ny * dysint
        zt = 0
        # подготовка исходных данных для вызова OrbitRange
        fizt = fizt0 + yt / Rz
        betazt = betazt0 + xt / Rz / math.cos(fizt0)
        Hz = zt
        fiztSh = 0
        betaztSh = 0
        # номер первого периода на интервале синтезирования
        qst = int(tauRli / Tr) + 1
        # аппроксимируем суммарную дальность фазовый центр антенны на
передачу -
        # земная точка - фазовый центр приемного канала на интервале
        # синтезирования полиномом третьей степени по пяти точкам
        rzt = cuda.local.array((3, 1), dtype=np.float64)
        RR = cuda.local.array(5, dtype=np.float64)
        for k in range(5):
            # получение координат фазового центра передающей антенны в НГЦСК
            rrsa = cuda.local.array((3, 1), dtype=np.float64)
            rrsa[0, 0] = XYZ_rsa_ts[qq[k], 0] * dxConsort
            rrsa[1, 0] = XYZ_rsa_ts[qq[k], 1] * dxConsort
            rrsa[2, 0] = XYZ_rsa_ts[qq[k], 2] * dxConsort
            # получение координат фазового центра приемного канала в НГЦСК
            rRch = cuda.local.array((3, 1), dtype=np.float64)
            rRch[0, 0] = XYZ_rsa_ts[qq[k], 6] * dxConsort
            rRch[1, 0] = XYZ_rsa_ts[qq[k], 7] * dxConsort
            rRch[2, 0] = XYZ_rsa_ts[qq[k], 8] * dxConsort
            # вычисление координат земной точки в НГЦСК
            rzt[0, 0] = (Rz + Hz) * math.cos(fizt + fiztSh * tt[k]) *
math.cos(
                2 * np.pi / Tz * (Tst + tt[k]) + betazt + betaztSh * tt[k])
            rzt[1, 0] = (Rz + Hz) * math.cos(fizt + fiztSh * tt[k]) *
math.sin(

```

```

        2 * np.pi / Tz * (Tst + tt[k]) + betazt + betaztSh * tt[k])
    rzt[2, 0] = (Rz + Hzt) * math.sin(fizt + fiztSh * tt[k])
    RR[k] = math.sqrt((rrsa[0,0] - rzt[0,0])**2+(rrsa[1,0] -
rzt[1,0])**2+(rrsa[2,0] - rzt[2,0])**2)+\
        math.sqrt((rRch[0,0] - rzt[0,0])**2+(rRch[1,0] -
rzt[1,0])**2+(rRch[2,0] - rzt[2,0])**2)
    # аппроксимация дальности полиномом третьей степени
    B0 = RR[0] + RR[1] + RR[2] + RR[3] + RR[4]
    B1 = RR[0] * tt[0] + RR[1] * tt[1] + RR[2] * tt[2] + RR[3] * tt[3] +
RR[4] * tt[4]
    B2 = RR[0] * tt[0] ** 2 + RR[1] * tt[1] ** 2 + RR[2] * tt[2] ** 2 +
RR[3] * tt[3] ** 2 + RR[4] * tt[4] ** 2
    B3 = RR[0] * tt[0] ** 3 + RR[1] * tt[1] ** 3 + RR[2] * tt[2] ** 3 +
RR[3] * tt[3] ** 3 + RR[4] * tt[4] ** 3
    A = cuda.local.array((4, 4), dtype=np.float64)
    A[0, 0], A[0, 1], A[0, 2], A[0, 3] = 5, sumtt, sumtt2, sumtt3
    A[1, 0], A[1, 1], A[1, 2], A[1, 3] = sumtt, sumtt2, sumtt3, sumtt4
    A[2, 0], A[2, 1], A[2, 2], A[2, 3] = sumtt2, sumtt3, sumtt4, sumtt5
    A[3, 0], A[3, 1], A[3, 2], A[3, 3] = sumtt3, sumtt4, sumtt5, sumtt6
    B = cuda.local.array((4, 1), dtype=np.float64)
    B[0, 0], B[1, 0], B[2, 0], B[3, 0] = B0, B1, B2, B3
    A_inv = cuda.local.array((4, 4), dtype=np.float64)
    inverse_matrix_cuda4(A, A_inv)
    pr = cuda.local.array((4, 1), dtype=np.float64)
    dot_matrix_cuda(A_inv, B, pr)
    pr1 = cuda.local.array((1, 4), dtype=np.float64)
    flip_and_transpose_cuda(pr, pr1)
    # вычисляем все дальности на интервале наблюдения

    # дальность на траверсе
    d0 = pr[0, 0]
    # ar=Vrsa^2/d0 # радиальное ускорение для компенсации МД и МЧ
    # некомпенсированные скорости для приемных каналов
    Vr1 = (Lrch / 2) / d0 * Vrsa
    # непосредственно суммирование - КН с компенсацией МД и МЧ
    sum1 = 0
    # дальность в момент начала синтезирования для первого канала
    for q in range(q1 - 1, q2):
        b = cuda.local.array((4, 1), dtype=np.float64)
        i = q - q1 + 1
        t1 = i * Tr
        b[0, 0] = t1 ** 3
        b[1, 0] = t1 ** 2
        b[2, 0] = t1
        b[3, 0] = 1.0
        # r Rch zt[0, i] =
        d = pr1[0, 0] * b[0, 0] + pr1[0, 1] * b[1, 0] + \
            pr1[0, 2] * b[2, 0] + pr1[0, 3] * b[3, 0]
        # дробный номер отсчета по быстрому времени
        ndr = (d / speedOfL - t_r_w + T0) * Kss * Fs - 1
        n = int(ndr)
        drob = ndr % 1
        ut = Uout01ss[n, q] * (1 - drob) + Uout01ss[n + 1, q] * drob
        fiq = 2 * math.pi / lamda * (d - q1)
        # суммируем с учетом сдвига РЛИ по скорости
        sum1 = sum1 + ut * WinSampl[q - q1 + 1] * e ** (-1j * fiq) * e **
(
            1j * 2 * np.pi * Vr1 / lamda * (q - q1 + 1) * Tr)

    Zxy1[nx, ny] = sum1

@cuda.jit
def kernel_2d_array_2(Zxy1, Zxy2, Nxsint, Nysint, Uout01ss, Uout02ss, dxsint,

```

```

dysint, fizt0,
                                Rz, betazt0, Tr, XYZ_rsa_ts, dxConsort, Tz, Vrsa,
tauRli, Inabl, qq, tt,
                                Lrch, speedOfL, t_r_w, Kss, Fs, lamda, WinSampl, e, T0,
                                sumtt, sumtt2, sumtt3, sumtt4, sumtt5, sumtt6, q1, q2,
Tst):
    nx, ny = cuda.grid(2)
    # координаты текущей точки наблюдения
    xt = (-(Nxsint - 1) / 2 + nx) * dxsint
    yt = (-(Nysint - 1) / 2 + ny) * dysint
    zt = 0
    # подготовка исходных данных для вызова OrbitRange
    fizt = fizt0 + yt / Rz
    betazt = betazt0 + xt / Rz / math.cos(fizt0)
    Hzt = zt
    fiztSh = 0
    betaztSh = 0
    # номер первого периода на интервале синтеза
    qst = int(tauRli / Tr) + 1
    # аппроксимируем суммарную дальность фазовый центр антенны на передачу -
    # земная точка - фазовый центр приемного канала на интервале
    # синтеза полиномом третьей степени по пяти точкам
    r_Rch_zt = np.zeros((2, Inabl))
    rzt = np.zeros((3, 1))
    RR = np.zeros(5)
    for irch in range(1, 3):
        for k in range(5):
            # получение координат фазового центра передающей антенны в НГцСК
            rrsa = np.array(
                [[XYZ_rsa_ts[qq[k], 0]], [XYZ_rsa_ts[qq[k], 1]],
                [XYZ_rsa_ts[qq[k], 2]]]) * dxConsort
            # получение координат фазового центра приемного канала в НГцСК
            if irch == 1:
                rRch = np.array(
                    [[XYZ_rsa_ts[qq[k], 6]], [XYZ_rsa_ts[qq[k], 7]],
                    [XYZ_rsa_ts[qq[k], 8]]]) * dxConsort
            if irch == 2:
                rRch = np.array(
                    [[XYZ_rsa_ts[qq[k], 9]], [XYZ_rsa_ts[qq[k], 10]],
                    [XYZ_rsa_ts[qq[k], 11]]]) * dxConsort
            # вычисление координат земной точки в НГцСК
            rzt[0, 0] = (Rz + Hzt) * math.cos(fizt + fiztSh * tt[k]) *
            math.cos(
                2 * np.pi / Tz * (Tst + tt[k]) + betazt + betaztSh * tt[k])
            rzt[1, 0] = (Rz + Hzt) * math.cos(fizt + fiztSh * tt[k]) *
            math.sin(
                2 * np.pi / Tz * (Tst + tt[k]) + betazt + betaztSh * tt[k])
            rzt[2, 0] = (Rz + Hzt) * math.sin(fizt + fiztSh * tt[k])
            RR[k] = np.linalg.norm(rrsa - rzt) + np.linalg.norm(rRch - rzt)
    # аппроксимация дальности полиномом третьей степени
    B0 = np.sum(RR)
    B1 = np.sum(RR * tt)
    B2 = np.sum(RR * (tt ** 2))
    B3 = np.sum(RR * (tt ** 3))
    A = np.array([[5, sumtt, sumtt2, sumtt3],
                  [sumtt, sumtt2, sumtt3, sumtt4],
                  [sumtt2, sumtt3, sumtt4, sumtt5],
                  [sumtt3, sumtt4, sumtt5, sumtt6]])
    B = np.array([B0], [B1], [B2], [B3])
    pr = np.linalg.inv(A).dot(B)
    pr1 = np.flip(pr.T)
    # вычисляем все дальности на интервале наблюдения
    for i in range(Inabl):
        t1 = i * Tr

```

```

        b = np.array([[t1 ** 3], [t1 ** 2], [t1], [1.0]])
        r_Rch_zt[irch - 1, i] = prl[0, 0] * b[0, 0] + prl[0, 1] * b[1, 0]
+ \
                                prl[0, 2] * b[2, 0] + prl[0, 3] * b[3, 0]

# дальность на траверсе
d0 = r_Rch_zt[0, 0]
# ar=Vrsa^2/d0 # радиальное ускорение для компенсации МД и МЧ
# некомпенсированные скорости для приемных каналов
Vr1 = (Lrch / 2) / d0 * Vrsa
Vr2 = (-Lrch / 2) / d0 * Vrsa
# непосредственно суммирование - КН с компенсацией МД и МЧ
sum1 = 0
sum2 = 0
# дальность в момент начала синтезирования для первого канала
d1 = r_Rch_zt[0, 0]
for q in range(q1 - 1, q2):
    d = r_Rch_zt[0, q - q1 + 1]
    # дробный номер отсчета по быстрому времени
    ndr = (d / speedOfL - t_r_w + T0) * Kss * Fs - 1
    n = int(ndr)
    drob = ndr % 1
    ut = Uout01ss[n, q] * (1 - drob) + Uout01ss[n + 1, q] * drob
    fiq = 2 * math.pi / lamda * (r_Rch_zt[0, q - q1 + 1] - q1)
    # суммируем с учетом сдвига РЛИ по скорости
    sum1 = sum1 + ut * WinSampl[q - q1 + 1] * e ** (-1j * fiq) * e ** (
        1j * 2 * np.pi * Vr1 / lamda * (q - q1 + 1) * Tr)

    ut = Uout02ss[n, q] * (1 - drob) + Uout02ss[n + 1, q] * drob
    fiq = 2 * np.pi / lamda * (r_Rch_zt[1, q - q1 + 1] - q1)
    sum2 = sum2 + ut * WinSampl[q - q1 + 1] * e ** (-1j * fiq) * e ** (
        1j * 2 * np.pi * Vr2 / lamda * (q - q1 + 1) * Tr)

Zxy1[nx, ny] = sum1
Zxy2[nx, ny] = sum2

@time_of_function
def gpu_detail_big_cycle1(Zxy1, Nxsint, Nysint, Uout01ss, dxsint, dysint,
fizt0,
                                Rz, betazt0, Tr, XYZ_rsa_ts, dxConsort, Tz, Vrsa,
tauRli, Inabl, qq, tt,
                                Lrch, speedOfL, t_r_w, Kss, Fs, lamda, WinSampl, e,
T0,
                                sumtt, sumtt2, sumtt3, sumtt4, sumtt5, sumtt6, q1,
q2, Tst):
    threads_per_block = (16, 16)
    blocks_per_grid_x = math.ceil(Zxy1.shape[0] / threads_per_block[0])
    blocks_per_grid_y = math.ceil(Zxy1.shape[1] / threads_per_block[1])
    blocks_per_grid = (blocks_per_grid_x, blocks_per_grid_y)
    kernel_2d_array_1[blocks_per_grid, threads_per_block](
        Zxy1, Nxsint, Nysint, Uout01ss, dxsint, dysint, fizt0,
        Rz, betazt0, Tr, XYZ_rsa_ts, dxConsort, Tz, Vrsa, tauRli, Inabl, qq,
tt,
        Lrch, speedOfL, t_r_w, Kss, Fs, lamda, WinSampl, e, T0,
        sumtt, sumtt2, sumtt3, sumtt4, sumtt5, sumtt6, q1, q2, Tst
    )
    return Zxy1

@time_of_function
def gpu_detail_big_cycle2(Zxy1, Zxy2, Nxsint, Nysint, Uout01ss, Uout02ss,
dxsint, dysint, fizt0,
                                Rz, betazt0, Tr, XYZ_rsa_ts, dxConsort, Tz, Vrsa,

```



```

tauRli, Inabl, qq, tt,
                                Lrch, speedOfL, t_r_w, Kss, Fs, lamda, WinSampl, e,
T0,
                                sumtt, sumtt2, sumtt3, sumtt4, sumtt5, sumtt6, q1,
q2, Tst):
    threads_per_block = (16, 16)
    blocks_per_grid_x = math.ceil(Zxy1.shape[0] / threads_per_block[0])
    blocks_per_grid_y = math.ceil(Zxy1.shape[1] / threads_per_block[1])
    blocks_per_grid = (blocks_per_grid_x, blocks_per_grid_y)
    kernel_2d_array_2[blocks_per_grid, threads_per_block](
        Zxy1, Zxy2, Nxsint, Nysint, Uout01ss, Uout02ss, dxsint, dysint,
fizt0,
        Rz, betazt0, Tr, XYZ_rsa_ts, dxConsort, Tz, Vrsa, tauRli, Inabl, qq,
tt,
        Lrch, speedOfL, t_r_w, Kss, Fs, lamda, WinSampl, e, T0,
        sumtt, sumtt2, sumtt3, sumtt4, sumtt5, sumtt6, q1, q2, Tst
    )
    return Zxy1

```

Файл вспомогательных CUDA функций *jit_functions.py*:

```

from numba import cuda
import numpy as np

@cuda.jit(device=True)
def dot_matrix_cuda(A, B, result):
    for t in range(result.shape[0]):
        result[t, 0] = 0.
    for i in range(A.shape[0]):
        for j in range(B.shape[1]):
            for k in range(A.shape[1]):
                result[i, j] += A[i, k] * B[k, j]

@cuda.jit(device=True)
def inverse_matrix_cuda3(matrix, inverse):
    # Получение размера матрицы
    n = 3

    # Проверка, что матрица квадратная
    if n != matrix.shape[1]:
        raise ValueError("Матрица должна быть квадратной")

    # Создание расширенной матрицы, добавляя единичную матрицу справа
    augmented_matrix = cuda.local.array((3, 6), dtype=np.float64)
    for i in range(n):
        for j in range(n):
            augmented_matrix[i, j] = matrix[i, j]
            augmented_matrix[i, j + n] = 1 if i == j else 0

    # Прямой ход метода Гаусса
    for i in range(n):
        # Находим ведущий элемент
        pivot = augmented_matrix[i, i]

        # Делим текущую строку на ведущий элемент
        for j in range(2 * n):
            augmented_matrix[i, j] /= pivot

    # Обнуляем остальные элементы в столбце
    for j in range(n):

```

```

        if j != i:
            factor = augmented_matrix[j, i]
            for k in range(2 * n):
                augmented_matrix[j, k] -= factor * augmented_matrix[i, k]

# Извлекаем обратную матрицу из расширенной матрицы
for i in range(n):
    for j in range(n):
        inverse[i, j] = augmented_matrix[i, j + n]

# Извлекаем обратную матрицу из расширенной матрицы
for i in range(n):
    for j in range(n):
        inverse[i, j] = augmented_matrix[i, j + n]

@cuda.jit(device=True)
def inverse_matrix_cuda4(matrix, inverse):
    # Получение размера матрицы
    n = 4

    # Проверка, что матрица квадратная
    if n != matrix.shape[1]:
        raise ValueError("Матрица должна быть квадратной")

    # Создание расширенной матрицы, добавляя единичную матрицу справа
    augmented_matrix = cuda.local.array((4, 8), dtype=np.float64)
    for i in range(n):
        for j in range(n):
            augmented_matrix[i, j] = matrix[i, j]
            augmented_matrix[i, j + n] = 1 if i == j else 0

    # Прямой ход метода Гаусса
    for i in range(n):
        # Находим ведущий элемент
        pivot = augmented_matrix[i, i]

        # Делим текущую строку на ведущий элемент
        for j in range(2 * n):
            augmented_matrix[i, j] /= pivot

        # Обнуляем остальные элементы в столбце
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j, i]
                for k in range(2 * n):
                    augmented_matrix[j, k] -= factor * augmented_matrix[i, k]

    # Извлекаем обратную матрицу из расширенной матрицы
    for i in range(n):
        for j in range(n):
            inverse[i, j] = augmented_matrix[i, j + n]

    # Извлекаем обратную матрицу из расширенной матрицы
    for i in range(n):
        for j in range(n):
            inverse[i, j] = augmented_matrix[i, j + n]

@cuda.jit(device=True)
def flip_and_transpose_cuda(pr, pr1):
    rows, cols = pr.shape

    # Копирование элементов в обратном порядке

```

```
for i in range(rows):  
    for j in range(cols):  
        pr1[j, i] = pr[rows - 1 - i, j]
```